Lecture 1.4 – Intro to R and R Studio

Specific Learning Objectives:

1.1.1 – Understand how to use the command line.

1.1.2 – Understand how to use the help function of R.

1.1.3 – Understand the basic syntax of the R language.

1.1.4 – Execute inbuilt mathematical functions to perform calculations in R.

1.1.5 – Learn how to assign variables.

1.1.6 – Understand the basic syntax of functions in R.

1.1.7 – Open, edit, and save a script in RStudio's editor.

2.2.1 – Create reproducible scripts in R.

About *R*

- A high-level interpreted language designed primarily to run statistical tests and visualize data.
    - Based on the language *S*, which is not used anymore.
    - Free, open-source language (meaning anyone can access the base code all the way down to compilers!)
    - Base *R* code is written in C, C++, Fortran, and *R*.
    - Interpreted language: meaning you don’t worry about compiling machine code, it happens automatically in the background.
    - *R* installations come with a simple command line and script editor.

About R Studio

- A Integrated Development Environment (IDE) for the *R* language.

  - R Studio is basically a wrapper with some helpful features! It is not *R* itself, which has to be downloaded separately.

  - Helpful features include:

    - Command-line prompt
    - Help viewer
    - Integrated plot editor
    - Jobs & testing environments
    - Memory management tools
    - File system navigator
    - Package Manager
    - Script Editor
    - Integrated RMarkdown & Knitr manager
    - Version Control panel

A Tour of R Studio

Script Editing Window

Environment variables

Current R Project

RStudio

Go to file/function | Addins | Project: (None)

install_dotwalkr.R

Source on Save | Run | Source

```
1  #### Install script ####
2
3  #### Required Packages ####
4  message("Testing for required packages, installing if not found.")
5  message(" ")
6  packages <- c("testthat", "R.matlab","data.table", "parallel", "doParallel",  "foreac
7
8  package.check <- lapply(
9    packages,
10   FUN <- function(x) {
11     if (!require(x, character.only = TRUE)) {
12       message(paste("Installing Package ", x," now."))
13       install.packages(x, dependencies = TRUE, repos='http://cran.us.r-project.org')
14       library(x, character.only = TRUE)
15     } else {
16       message(paste("Package ", x," found."))
17     }
18   }
19 )
20 message("Done with required packages!")
21 message(" ")
22
```

1:1 | Install script | R Script

Console ~/

```
Platform: x86_64-apple-darwin17.0 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

  Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

>
```

Environment | History | Connections | Tutorial

Import Dataset | List

Global Environment

Environment is empty

Files | Plots | Packages | Help | Viewer

New Folder | Delete | Rename | More

Home > Dropbox (Chapman) > dotwalkR

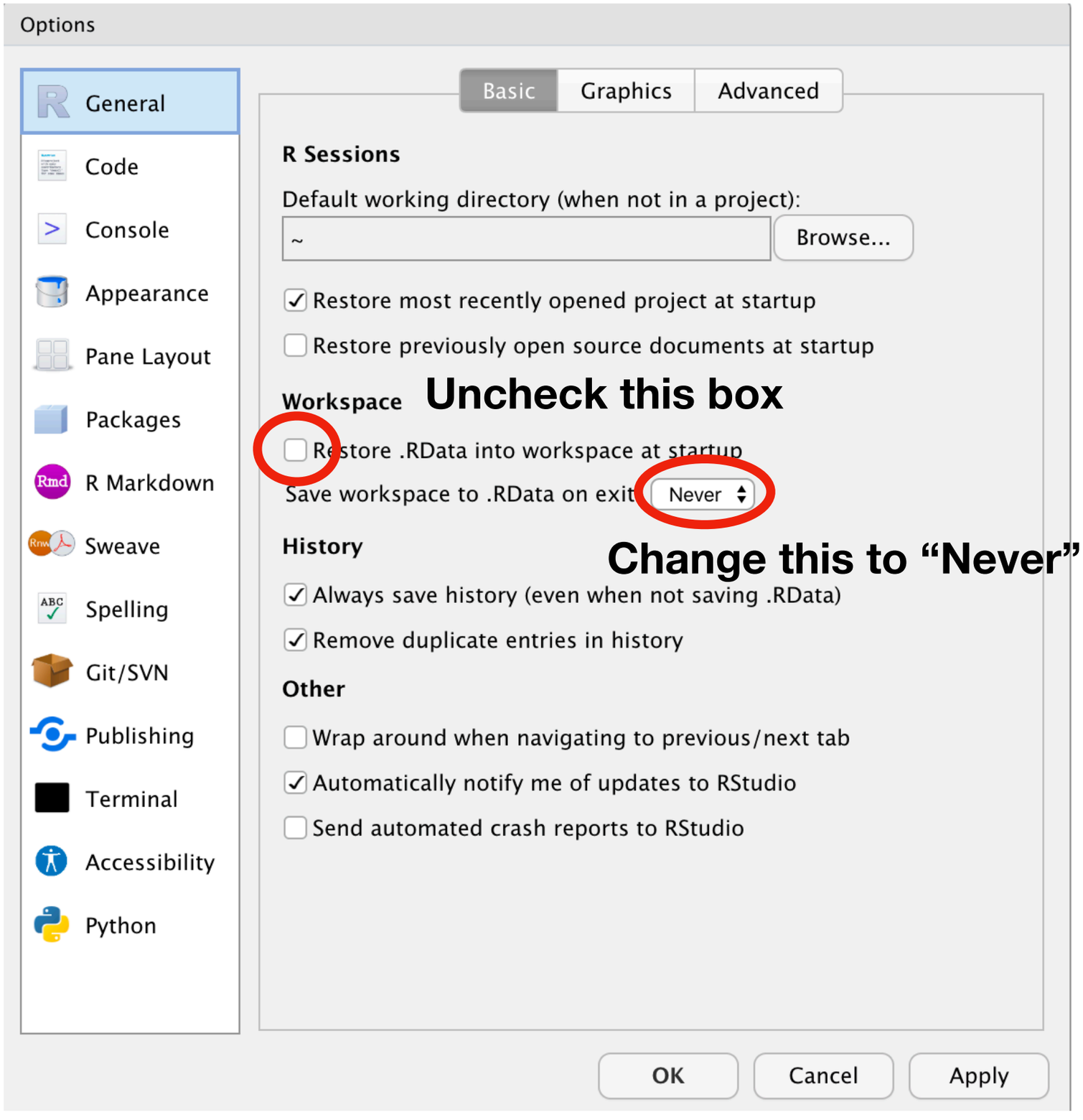
Name	Size	Modified
..		
.gitignore	50 B	Jan 15, 2021, 6:42 PM
.Rhistory	21.9 KB	Jan 27, 2021, 3:47 PM
data		
doc		
dotwalkR.Rproj	205 B	Jan 27, 2021, 3:47 PM
install_dotwalkr.R	808 B	Jan 25, 2021, 3:51 PM
README.md	7.7 KB	Jan 25, 2021, 4:46 PM
results		
Rplot.pdf	5.6 MB	Jan 22, 2021, 10:06 AM
Rplot01.pdf	513.5 KB	Jan 23, 2021, 8:15 AM
src		
tests		

File system navigator

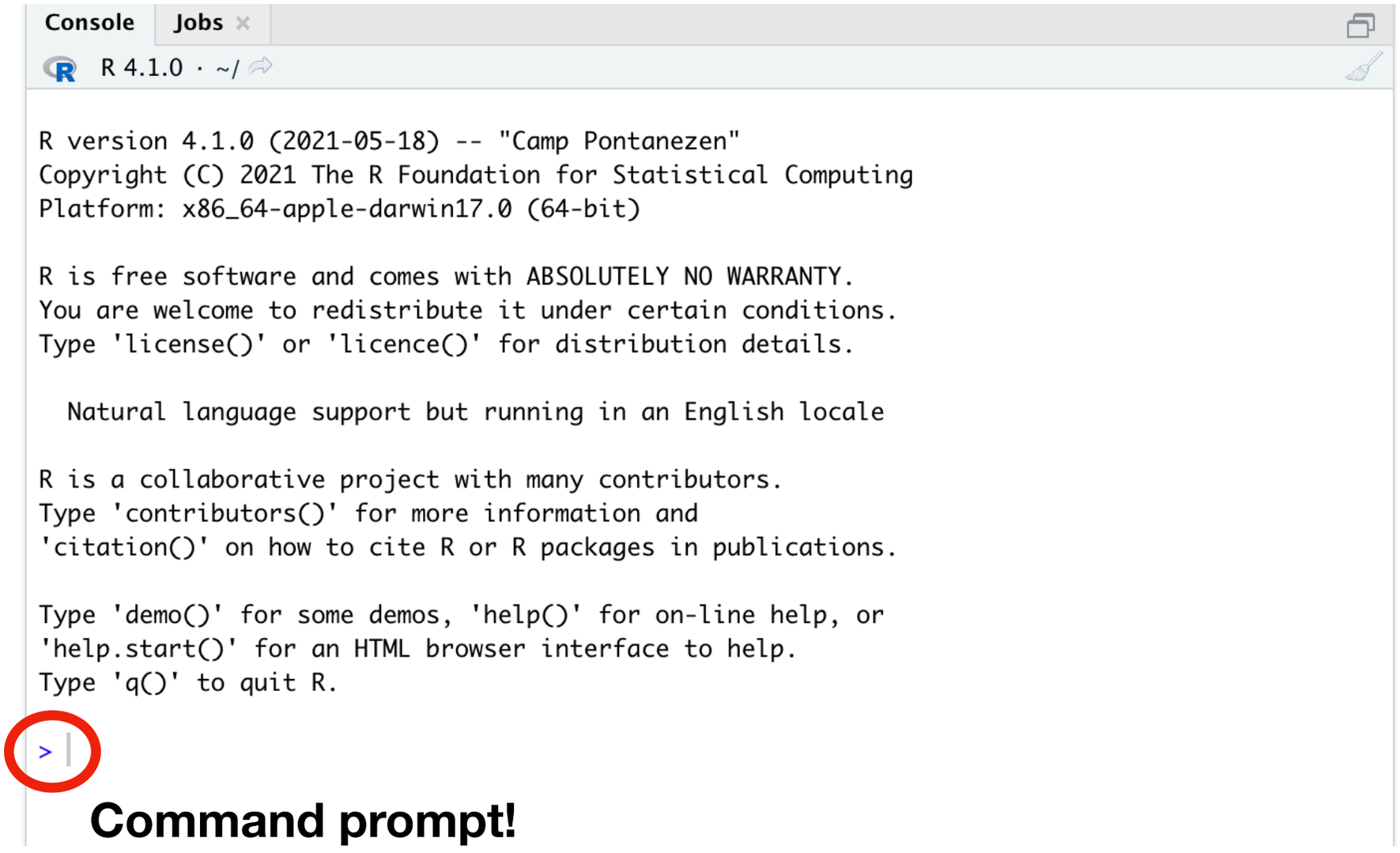
Command-line console

Adjusting Preferences

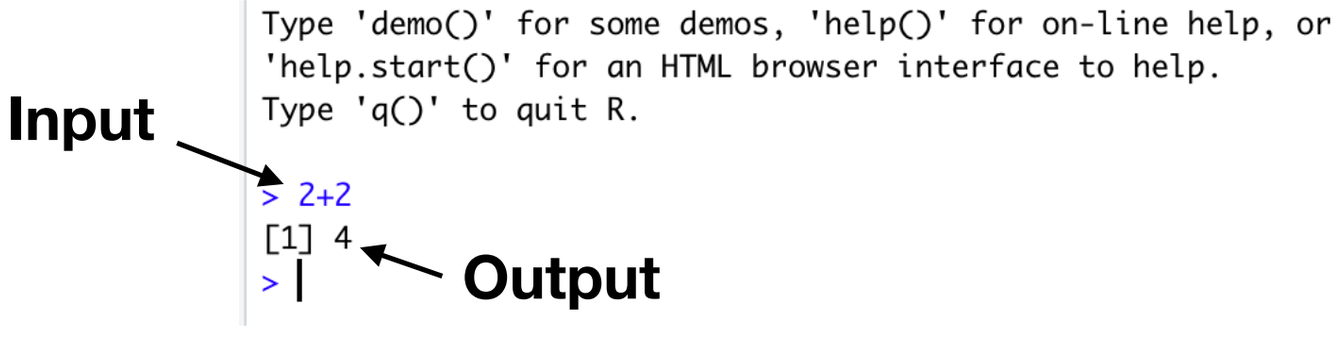
- Open RStudio's preferences:
  - Top bar > Tools > Global Options…
- We don't want your workspace saved!!!



Using Command Line



- Put your cursor here and enter something!



***R* can function as a basic calculator!**

R as a calculator

- R can act as a basic and scientific calculator

  - Addition and Subtraction

```
> 2+2
[1] 4
> 2-2
[1] 0
```

  - Multiplication and Division

```
> 2*2
[1] 4
> 2*9
[1] 18
```

```
> 9/3
[1] 3
> 14/7+2
[1] 4
> 14/(7+2)
[1] 1.555556
```

  - Exponents and Logarithms

```
> 5^2
[1] 25
> 6^-1
[1] 0.1666667
```

```
> log(234)
[1] 5.455321
> log(243,base=3)
[1] 5
```

It's a function!!

  - e notation

```
> 1e-8
[1] 1e-08
> 8.39284e5
[1] 839284
> 83792159740178234 6
[1] 8.379216e+17
```

Syntax in R

- Syntax is the set of rules that define what various combinations of symbols mean.
    - Each computing language has a different set of rules of how you can arrange symbols to tell the computer what to do.

```
> 2+2
[1] 4
> 2 + 2
[1] 4
> 2        +            2
[1] 4
> 22+
+
+ 2
[1] 24
> 2,2,+
Error: unexpected ',' in "2,"
>
```

You can put spaces and tabs in between elements

But the elements need to be in the correct order!

And they must make sense to the computer (no commas allowed here!)

A lot of your errors, especially at first, will be syntax errors. This is ok, just fix it and try again!

Let’s just make R produce errors

- We’re going to take a quick break to make R produce some errors. I make R errors all the time, constantly, even if it might look like I always know what I’m doing!

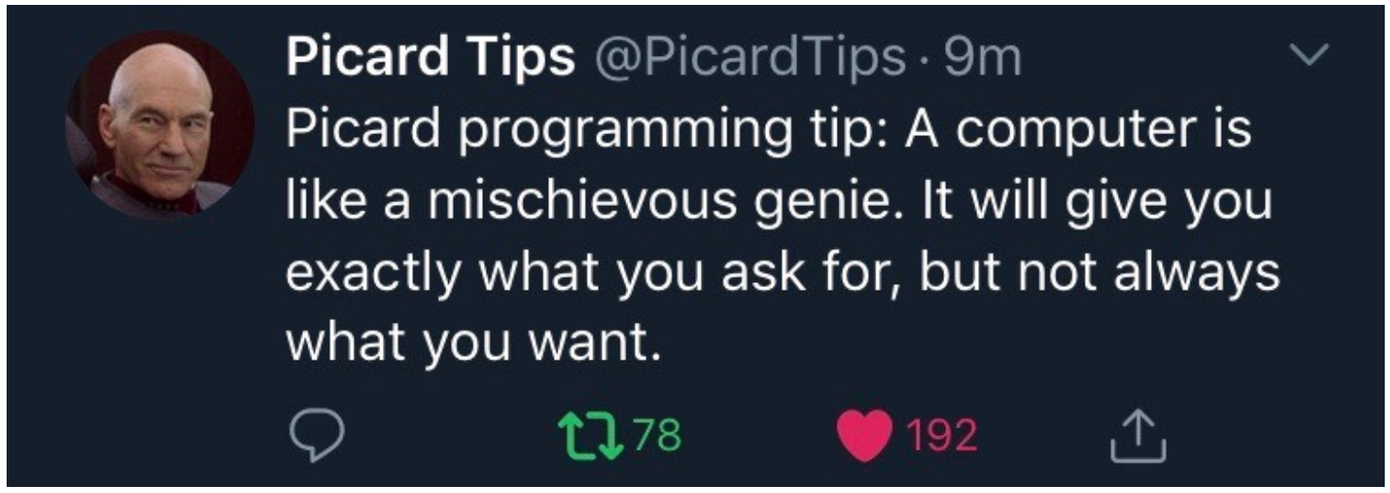
- Desensitize yourself to errors, they really aren’t a big deal. You’re not going to break anything.

- Feel free to swear at R, I don’t care and neither does it.

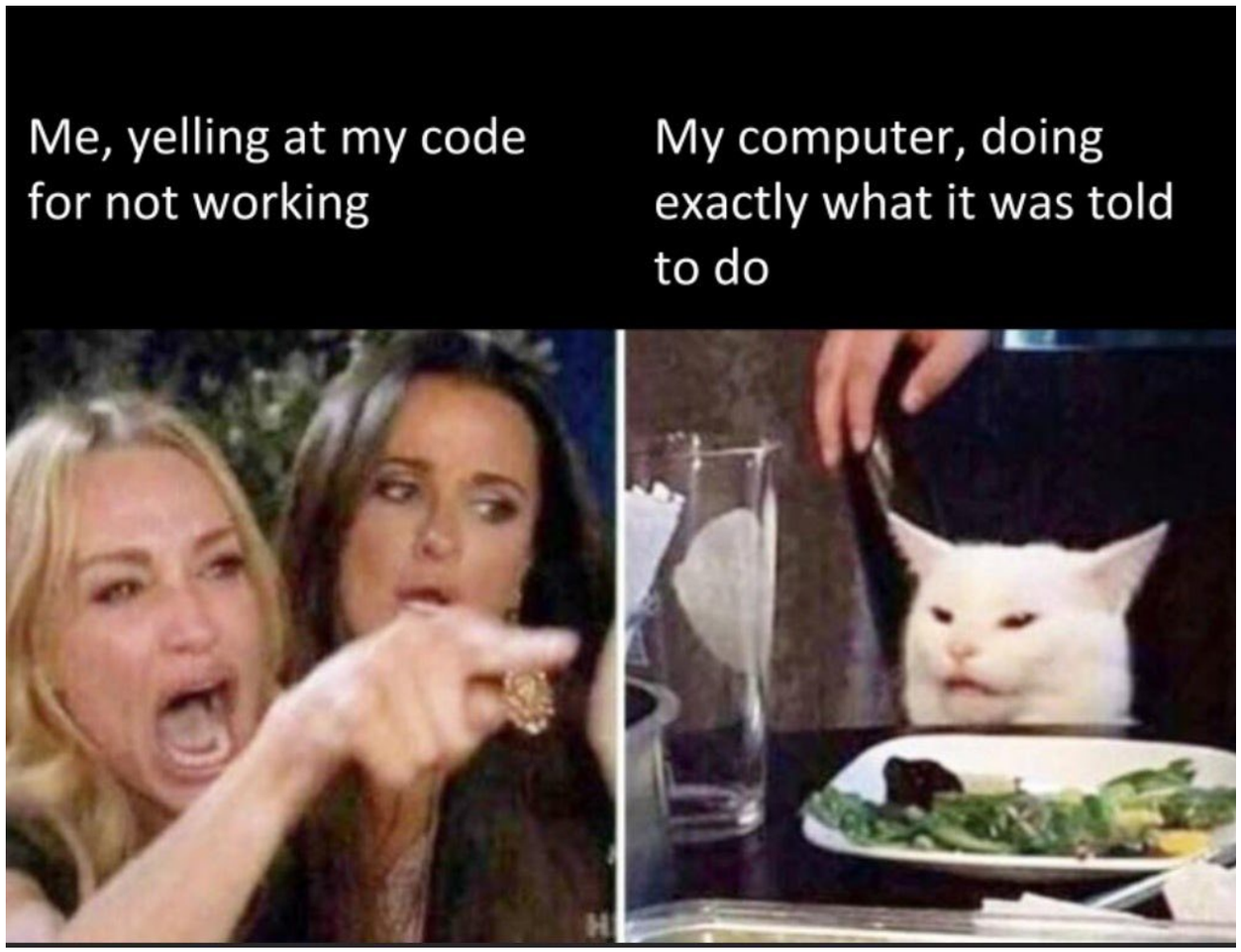
- A huge part of learning to code is learning to tolerate being frustrated! It’s ok and totally normal. Just don’t let it consume you, take a break! Do something else! You’ll get it eventually!

Some tips for using *R*

- **Remember that computers do EXACTLY what you tell them to, but that's often not what you THINK you are telling them to do!**



- **Computers really only do what you tell them to do. It's always your fault. (It's always my fault too.)**

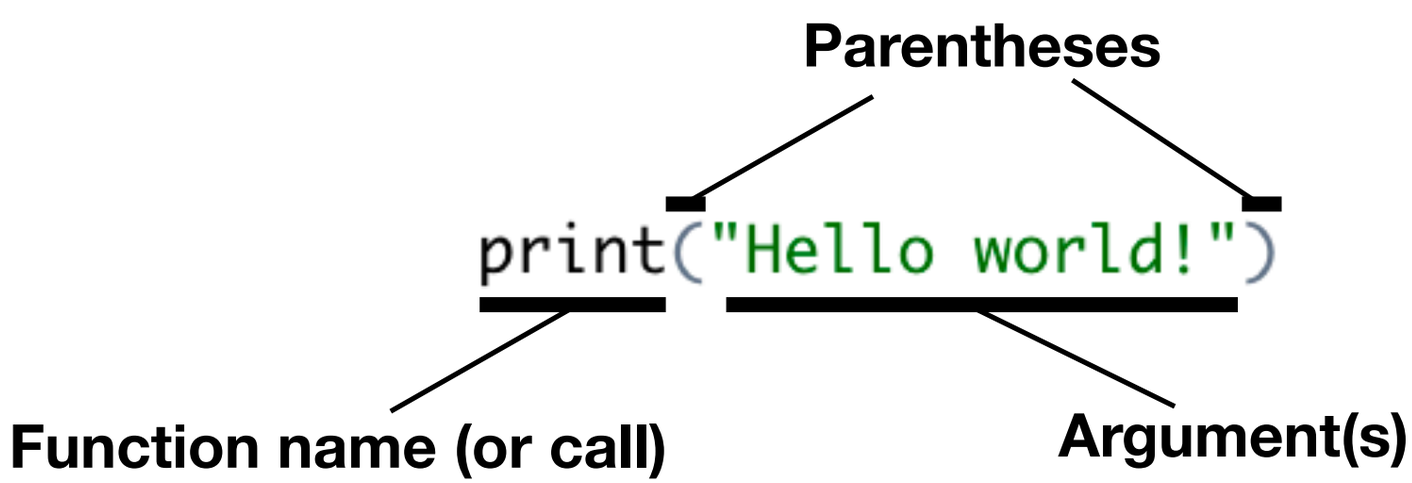


A lot of your errors, especially at first, will be syntax errors. This is ok, just fix it and try again!

Functions in R

- Functions reference other bits of code that you can run by “calling” it (written by someone else or you)

- Syntax of a function in *R*:



- **Function name (or call)**: the name of the function you desire to use, or how to “call” the function.

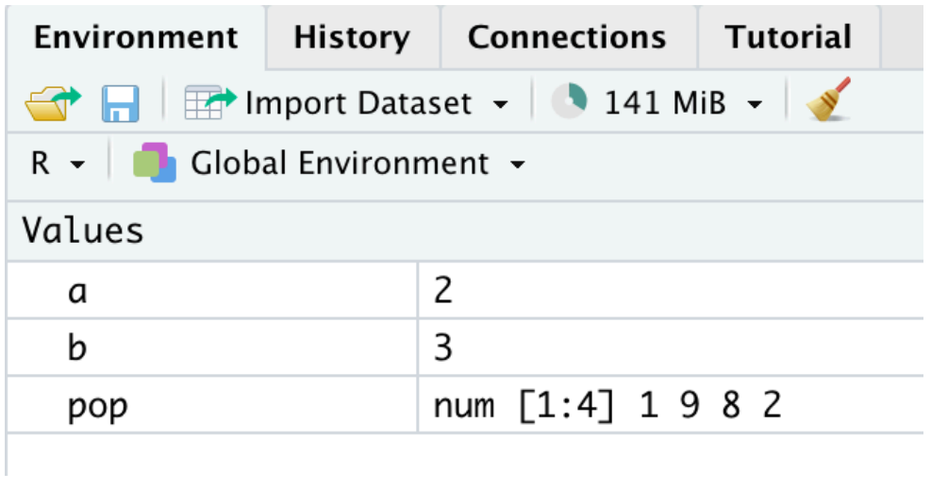
- **Parentheses**: Parentheses after the call is how R knows you want to call a function. You MUST put parentheses, even if the function takes no arguments, or else R will think you are trying to recall a variable!

- **Arguments**: options that you’d like to pass to the function.

Functions in R

- Use a function to create a vector:

```
> pop <- c(1, 9, 8, 2)
```



- Use another function to calculate the mean:

```
> mean(pop)
[1] 5
```

- You can nest functions within each other:

```
> mean(c(1, 9, 8, 2))
[1] 5
```

How many arguments a function can take is determined by the function!

- Functions can have many arguments, separated by commas:

arg 1 **arg 2** **arg 3**

```
> blep <- seq(from = 1, to = 10, by = 0.5)
```

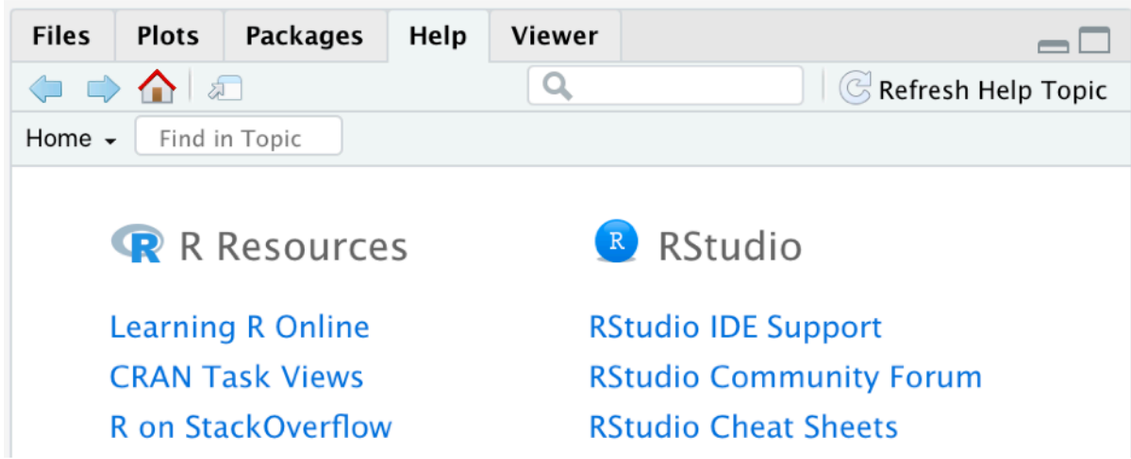
Creates a vector that is a sequence of numbers from 1 to 10 by 0.5

R has lazy evaluation!

```
> blep <- seq(1, 10, 0.5)
```

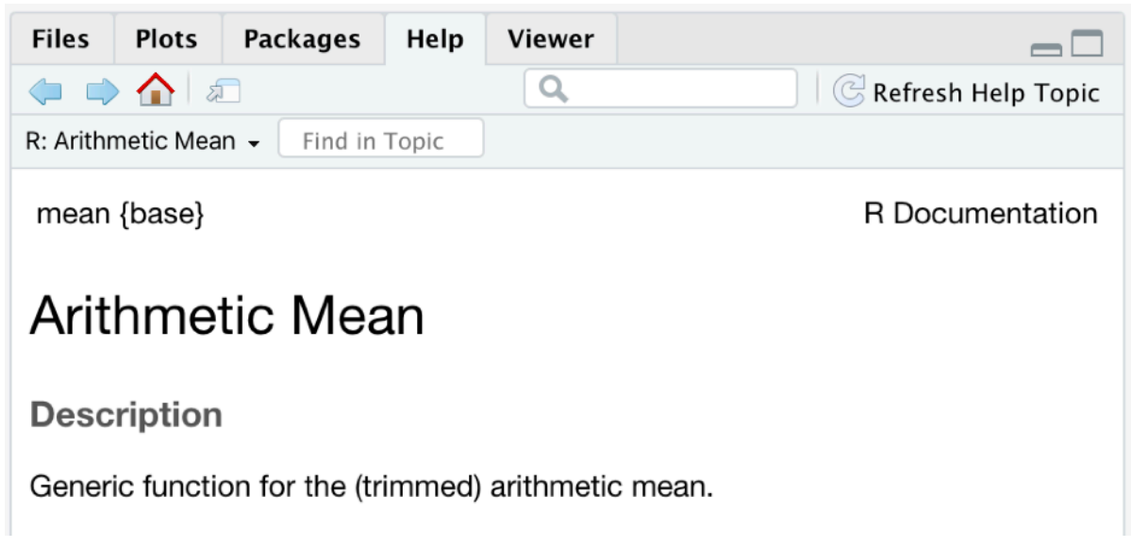
When things go sideways… use help!

- Help documentation exists for each function in R.
  - Help viewer is in the lower right-hand corner of RStudio as a tab.



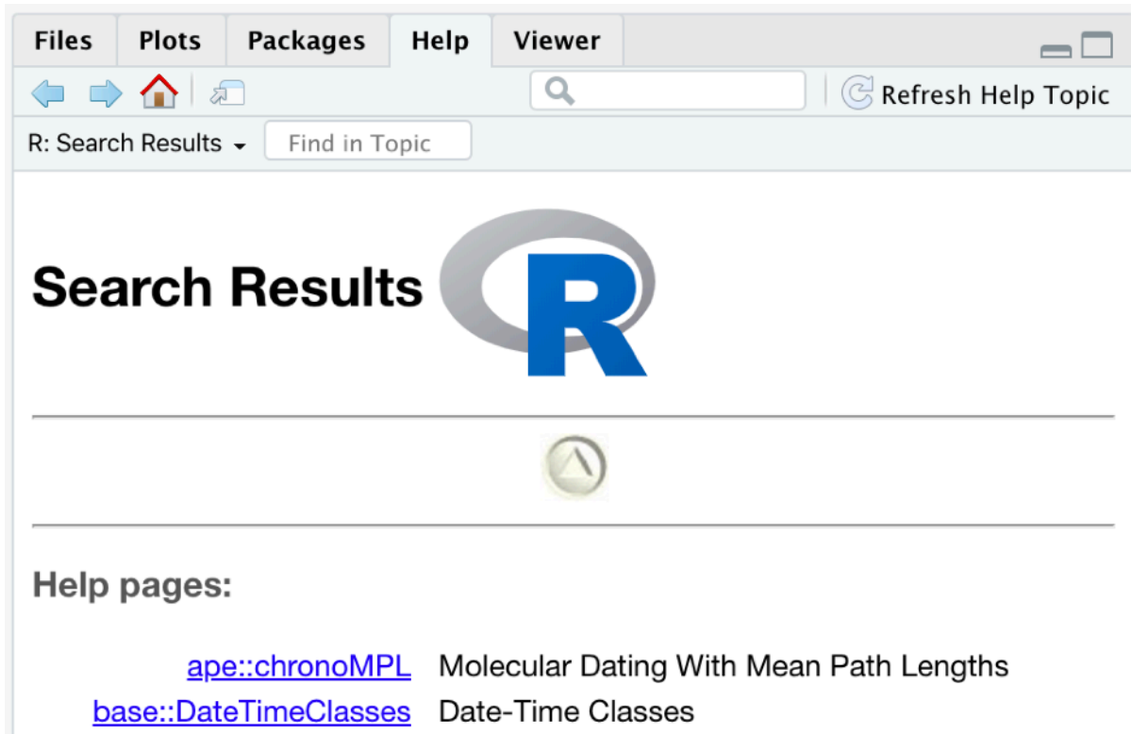
  - Access help documentation for a function:

```
> ?mean
```



- Do a keyword search:

```
> ??mean
```



Check Your Understanding

Have R calculate the square root of 164956 using the `sqrt()` function. What is the correct output?

Correct answer

a) `[1] 406.1478`

b) `[1] 12.8841`

c) `Error: unexpected input in "√"`

d)
```
Error in sqrt(164956, 2) : 2
arguments passed to 'sqrt' which
requires 1
```

Error in c: I used the sign √164956, even though I understand what this means, R doesn't understand so returns an error.

Error in d: I put too many numbers (or arguments) in the `sqrt()` function so it's telling you you need one argument only!

Assigning variables and objects

- You'll notice that when you add 2+2, R will give you an output, but it doesn't save or "store" that value anywhere. (Look at your environment, it is empty!)

- Assigning variables (objects): two methods

```
> a <- 2
```

- Assign arrow (less than sign and dash)

```
> b = 3
```

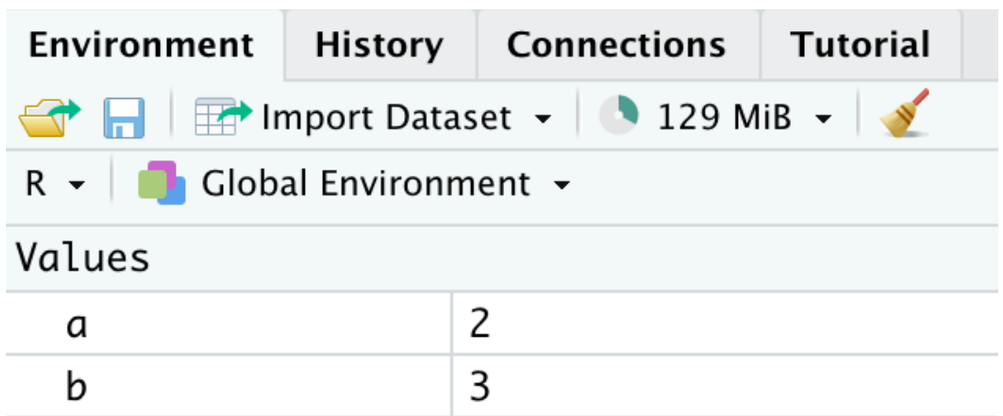
- Equal sign

For historical reasons, this is the preferred method for variables…

…and this is the preferred method for arguments inside functions.

Your environment tells you what's in R's memory! This is empty when R restarts!!

- Assigned variables appear in your environment!



Check Your Understanding

Syntax is important here! Using a space between the < and - will change the meaning of the command.

Try these: do they all have the same result? Why or why not?

a) `x<-2` b) `x <- 2` c) `x< - 2`

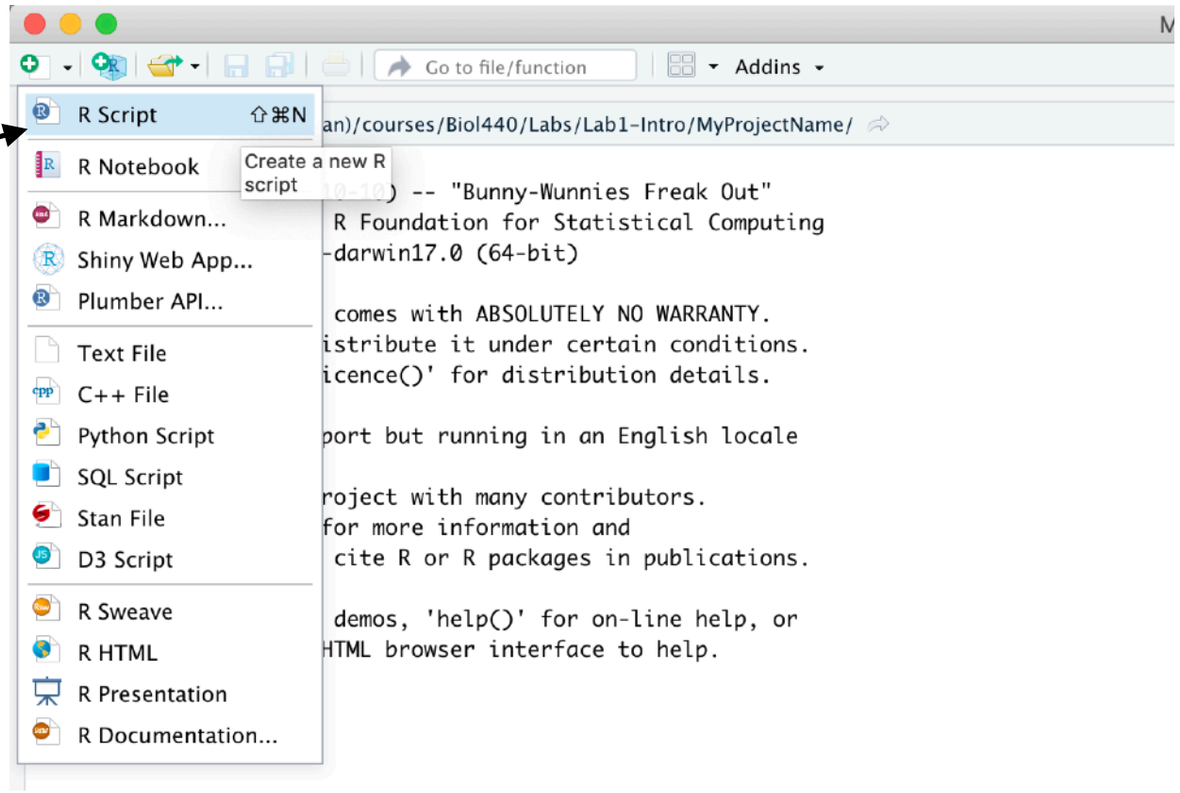
Does it matter which way the arrow points?

Try these: which work and which produce errors?

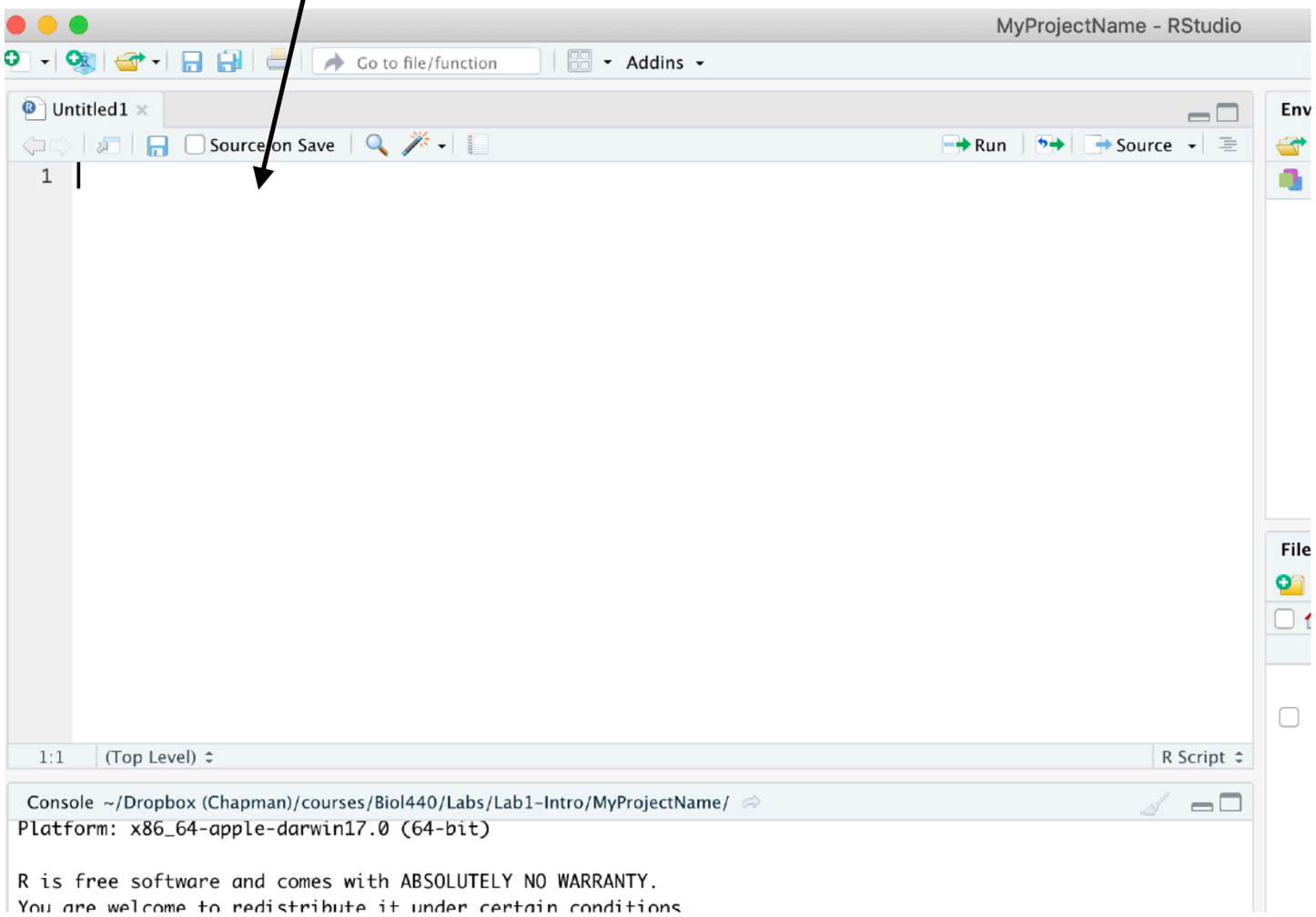
d) `7 -> j` b) `7 <- j` c) `j -> 7`

Your First R Script

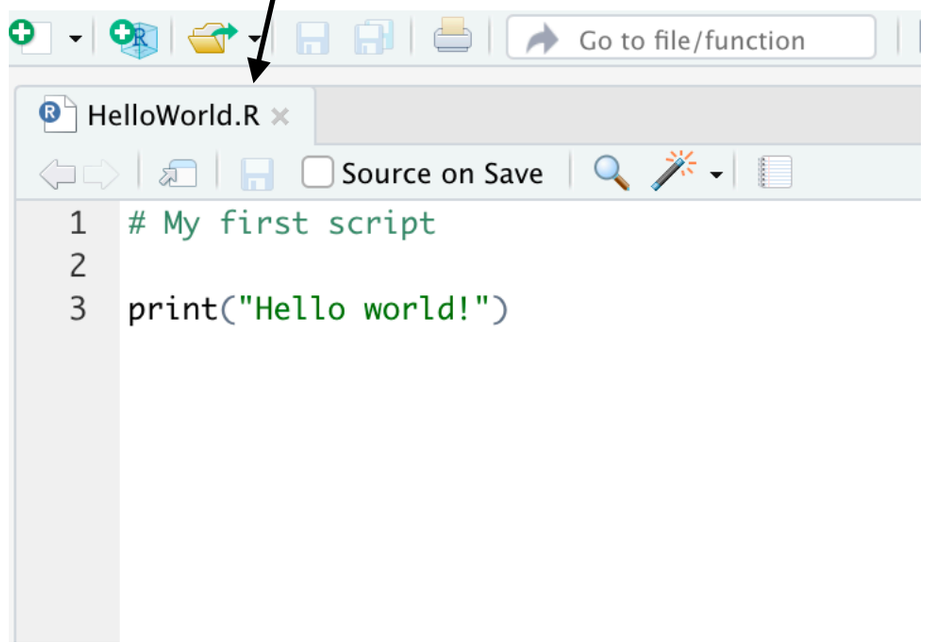
Click on “New File” tab the in upper left and select “R Script”.



This will open a window in the script editor. You’re ready to code!



Be sure to save your script with a name that ends in .R



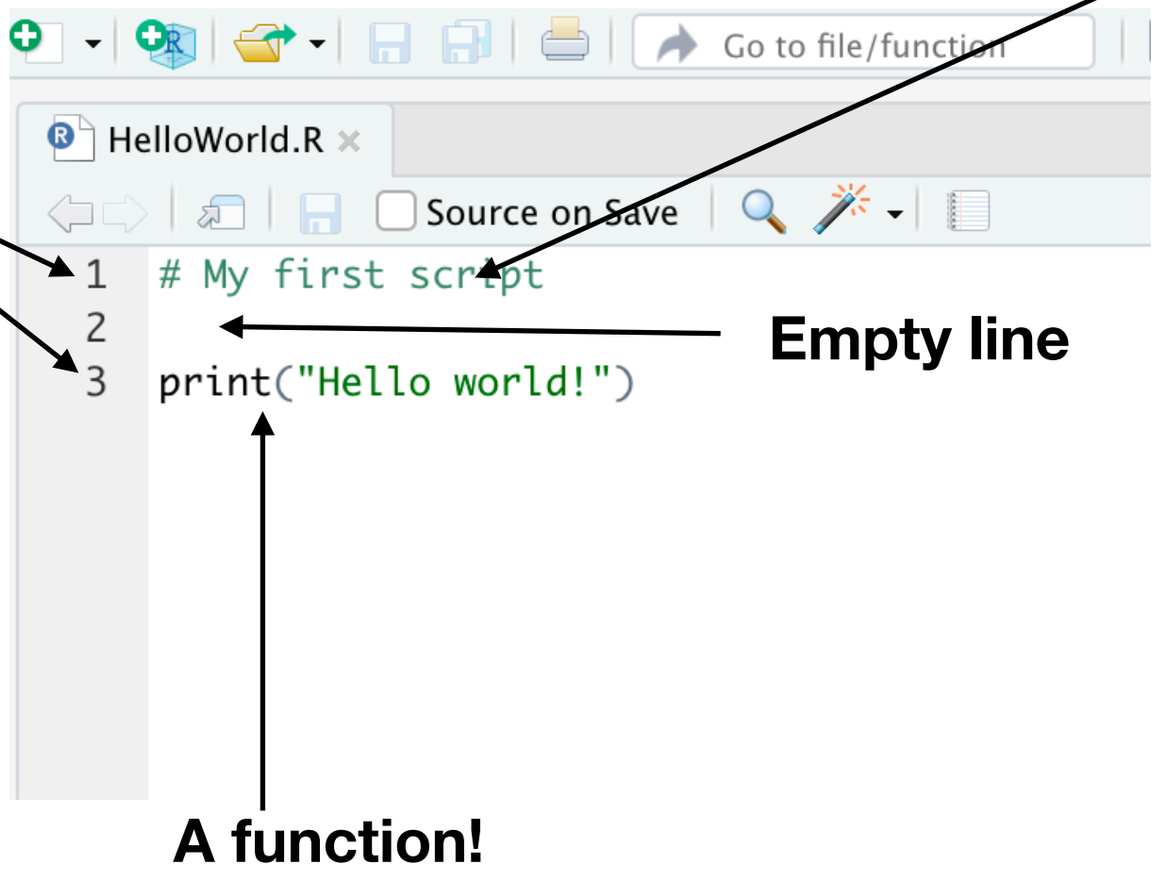
Your First R Script

A "comment"

- Anything after a pound symbol #
- Skipped by R (not evaluated)
- Appears in green in script editor
- Really useful for documenting work

Line numbers

- Useful for referencing specific bits of code
- Will show up in error messages

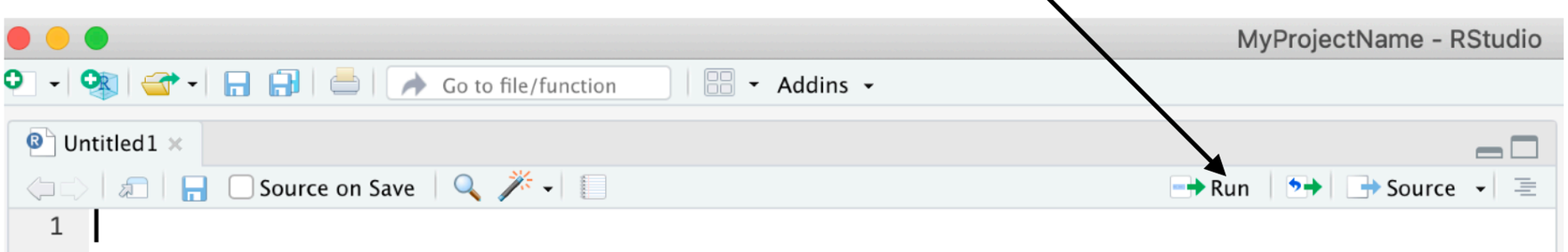


Empty line

A function!

How to execute code in a script:

- Run each line by clicking the "Run" button
- Use a shortcut:
  - Windows: Clt + Enter
  - Mac: Command + Enter



Functions in R

Check your understanding!

What happens if you enter the numbers into mean without the **c( )**? Or like this:

```
> mean(1, 9, 8, 2)
```

Why does it do this?

Functions in R

Check your understanding!

What happens if you enter the numbers into mean without the **c( )**? Or like this:

```
> mean(1, 9, 8, 2)
```

Why does it do this?

Why would the mean not calculate correctly based on the help documentation for **mean( )**?

More R Tips

- Use scripts to document code used during lecture! Make comments about what does and doesn't work, why errors occur, etc.

- Avoid using command line when possible, put it in a script!

- Make sure to write code that's evaluated from top to bottom without errors!

- You will probably get pretty frustrated. That's fine, just put it down and try again later.

- **Just because you get frustrated or produce errors from time to time doesn't mean you are bad at coding!**

Action Items

1. **Complete Assignment 1.5**

2. **Read Davies Chapters 3, 4, and 6 for next time.**